

In the **Hyper-Vibe Coding Course**, **Module 1**, titled **"🌱 Your First Vibe"**, is a beginner-level introduction focused on environment setup and the "no-fluff" philosophy of the ecosystem [1]. It is designed to move neurodivergent learners past the "freezing" point caused by traditional instructions by providing immediate, hands-on success [2].

Core Objectives

The primary goal is to establish a **"living cognitive architecture"** on the student's local machine [3, 4]. This involves transitioning from a blank setup to a fully operational 32-container AI infrastructure lab [4, 5].

Key Technical Tasks

Students perform the following steps to get their environment "autonomously alive":

- * **Infrastructure Installation:** Installing **Docker Desktop (Version 24+)** as the primary engine for the stack [6, 7].
- * **Repository Management:** Cloning the **HyperCode V2.4** repository to their local machine [8].
- * **Environment Configuration:** Setting up the **`.env`** file using a provided template to manage API keys and system secrets [8].
- * **Stack Deployment:** Running the command **`docker compose up -d`** to launch all 32 containers simultaneously [8].
- * **Interface Access:** Verifying the installation by accessing **Mission Control** (port 8088), the **BROski Terminal** (port 3000), and other core agent interfaces [8].

Gamification and Initial Wins

Module 1 introduces students to the **BROski\$ token system**, which is designed to provide "constant small wins" to maintain hyperfocus [9]. By completing the initial setup tasks and their first daily login, students begin earning:

- * **XP:** Used to climb the 7 tiers of the level system, starting as a **BROski Recruit** [10].
- * **Coins:** Awarded for actions like creating tasks (+2) or completing the daily login (+5) [10].
- * **Achievements:** Students may unlock **"First Blood"** (+50 XP, +20 Coins) upon completing their first setup task [10].

Essentially, Module 1 is about **stopping the "bureaucracy" of development** and giving the student control over an integrated swarm of agents right from the start [2, 3].

In **Module 2: Prompt Like a Pro**, the focus shifts from infrastructure setup to mastering the core communication method of the Hyperfocus Zone: **Natural Language Programming**. This module is designed to help students treat natural language as a high-level coding language to drive the platform's AI engine [1, 2].

Core Mission: The "Vibe Coding" Workflow

The module is built around the philosophy of **"Natural language → AI code → shipped product"** [1]. It aims to eliminate the "gatekeeping" and "fluff" of traditional programming by teaching students how to bypass "syntax bureaucracy" and focus on high-level system design [1, 3].

Learning Objectives and Key Skills

Based on the curriculum and the ecosystem's design, Module 2 covers:

- * **Mastering the AI Engine:** Students learn to communicate effectively with the system's core cognitive models—**Claude AI** for reasoning and **Ollama** for local model execution [Artifact 5].
- * **Prompt Engineering for Production:** Learning to write precise instructions that result in functional, deployable code rather than just snippets [1].
- * **System Pattern Recognition:** Using AI to translate the complex patterns of the 32-container stack into understandable concepts that "click" for the individual student [3, 4].
- * **Reducing "Instruction Freeze":** Developing techniques to break down large, overwhelming tasks into small, manageable prompts, which is specifically designed to help neurodivergent learners maintain hyperfocus [4, 5].

The Neurodivergent Advantage

Module 2 is specifically tailored for **ADHD, dyslexic, and autistic minds** who may find traditional documentation scattered or confusing [3, 4, 6].

- * **Pattern-Based Learning:** Instead of memorizing syntax, students are encouraged to use their natural ability to see "whole systems in patterns" [3].
- * **No Judgment, Just Clarity:** The AI acts as a "ride or die" partner, providing a safe environment where students can ask for help or clarification multiple times without the stress of traditional learning environments [4, 6].
- * **Constant Feedback:** By using natural language to get immediate code results, students receive the "constant small wins" necessary to stay in a state of hyperfocus [5, 7].

Essentially, Module 2 teaches you to **stop apologizing for your brain** and start using it as a superpower to build at the speed of thought [3, 6].

Would you like me to expand Module 2 into a full lesson plan with step-by-step tasks and a mini-project, similar to what we did for Module 1?

Module 3: Build Your First App marks a significant transition in the Hyper-Vibe Coding Course, moving you from foundational "Vibe Coding" into the **Beginner → Mid** level of full-stack development [1]. This is where the theoretical patterns you've learned start becoming tangible software.

The Mission: From Prompt to Prototype

The core objective of this module is to master the **"Natural Language → AI Code → Shipped Product"** workflow [2]. Instead of just running scripts or chatting with agents, you are building a functional, full-stack application from scratch without the traditional "syntax bureaucracy" that often causes neurodivergent developers to freeze [3, 4].

Key Skills & Technical Focus

In this module, you bridge the gap between frontend and backend systems:

- * **Frontend Foundations:** You begin working with **Next.js**, the same framework used for the Mission Control dashboard in the HyperCode ecosystem [5, 6].

* **Backend Integration:** You learn how to connect your interface to a **FastAPI backend**, which serves as the "FastAPI backbone" for memory and context management in the larger system [5, 6].

* **Rapid Iteration:** The focus is on **fast feedback loops**—using AI to generate code, testing it immediately in your local environment, and iterating based on the results [7].

The "Mid-Level" Leap

While Modules 1 and 2 focus on setup and communication, Module 3 introduces more complex architectural concepts:

* **Connecting the Dots:** You move beyond single-file scripts to understanding how a frontend "talks" to a backend API.

* **Ecosystem Alignment:** You start using tools that align with the **HyperAgent-SDK**, preparing you to eventually build agents that can be deployed across Discord or standalone backends [8, 9].

* **Zero Gatekeeping:** True to the course's "no-fluff" philosophy, you aren't forced to learn every line of CSS or SQL manually; instead, you learn to **direct the vision** while the AI handles the boilerplate building [2, 3].

Why This Module Matters for Hyperfocus

Module 3 is designed to provide the **"constant small wins"** necessary for ADHD and autistic minds to maintain momentum [10]. By the end of this module, you won't just have a collection of files; you will have a **working application** that you can see, touch, and interact with on your local machine, which serves as a major milestone toward becoming a **BROski Legend** in the ecosystem [11].

Would you like me to generate a detailed lesson plan for Module 3, including a specific mini-project you'd build at this stage?

In the **Hyper-Vibe Coding Course**, **Module 4: Full Stack Vibe** is an **intermediate-level** course where students move beyond simple prototypes and start building the "soul" of their applications using a robust database and authentication layer [1].

According to the sources and the curriculum architecture, here is what happens in Module 4:

1. Mastering the Supabase Backbone

While earlier modules focus on local development, Module 4 introduces **Supabase** as the primary backend-as-a-service [2]. Students learn to manage:

* **Database & Auth:** Implementing real-world authentication, including **password validation** features [1, 2].

* **Edge Functions:** Writing and deploying serverless functions, such as the **token sync edge functions** that keep the BROski\$ economy running [3].

2. Deepening Integration

The module focuses on wiring the different parts of the "Hyper Way" stack together. This includes:

- * **Vercel AI Gateway:** Learning how to route AI interactions through the **Vercel AI Gateway** via specific API routes (like the BROski chat route) [2, 3].
- * **Database Management:** Understanding how to use **PostgreSQL** (via Supabase) to store persistent agent data and student progress [2, 3].

3. Key Skills Unlocked

By working through this module, students develop the skills necessary to reach **Level 2 (Vibe Coder)** and prepare for **Level 3 (Agent Builder)** in the Agent Unlock System [4].

Key skills include:

- * Architecting **Supabase agents** with specialized skills [4].
- * Managing **Postgres best practices** within an AI-driven environment [3].
- * Syncing user data (like BROski\$ tokens) between different parts of the ecosystem [3].

The "Full Stack" Objective

The goal of Module 4 is to ensure that a student's vision is no longer just a "demo" but a **live, secure, and data-driven platform** [2]. It provides the technical foundation needed for **Module 5**, where students begin building the complex, self-healing agent swarms of the core HyperCode V2.4 engine [1, 5].

Would you like me to create a detailed lesson plan for Module 4, similar to the one we did for Module 1?

Module 5: 🔥 HyperCode The Hyper Way is the transition into the **Advanced** level of the Hyper-Vibe Coding Course [1]. In this module, students move beyond building standalone applications to mastering the **"living cognitive architecture"** of the HyperCode V2.4 engine [2, 3].

Core Objectives

The primary focus is understanding and directing a **swarm of AI agents** that autonomously design, deploy, heal, and evolve themselves within a complex Docker-based infrastructure [3]. Students shift from being "coders" to becoming **Meta-Architects** who direct the vision while the system handles the building [3].

Key Technical Components

In this module, you dive deep into the specific agents that make the ecosystem "autonomously alive":

- * **Agent X (The Meta-Architect):** Learning how this agent autonomously designs and deploys new specialized agents [4, 5].
- * **The Healer Agent:** Implementing the logic that allows the system to monitor itself and **auto-recover failed services** without human intervention [4, 6].
- * **Crew Orchestrator:** Mastering the lifecycle management and mission execution for the entire agent swarm [4, 7].
- * **The Evolutionary Pipeline:** Setting up the infrastructure that allows agents to **upgrade themselves autonomously** [5, 7].

Advanced Infrastructure Skills

Module 5 requires students to manage the full technical weight of a production-grade AI lab:

- * **32-Container Management:** Students learn to orchestrate a massive local stack including Redis, PostgreSQL, and specialized AI runners [6, 8].
- * **SRE Observability:** Wiring the full observability stack—Prometheus, Grafana, Loki, and Tempo—to monitor the "heartbeat" of the agents in real-time [6, 8].
- * **FastAPI Backbone:** Managing the core API that handles memory, context, and the MCP (Model Context Protocol) gateway [4, 7].

The Neurodivergent Edge

Consistent with the course philosophy, this advanced module is designed to reduce the "bureaucracy" of complex systems [3]. It teaches students to **see whole systems in patterns** rather than getting lost in syntax, allowing them to maintain hyperfocus while building high-level autonomous systems [3, 9].

By the end of Module 5, a student has graduated from building tools to **architecting a self-evolving empire**, setting the stage for the final "Ship & Scale" phase of the curriculum [1].

Would you like a detailed lesson plan for Module 5, including a project focused on building your own self-healing logic?

In the **Hyper-Vibe Coding Course**, **Module 6**, titled **" Ship & Scale,"** is the final and most advanced level of the curriculum, designated as **Hyper-Pro** [1]. It represents the culmination of the "Vibe Coding" journey, where you transition from a local development environment to a production-ready, scalable AI architecture [2, 3].

According to the sources, the following key activities and learning objectives define Module 6:

1. Production Economics (Stripe & Tokens)

The core focus of this module is building a real-world economy for your agents. You will learn to:

- * **Integrate Stripe Payments:** Implement end-to-end **Stripe Checkout** to handle real currency for token pack purchases (Starter, Builder, and Hyper packs) [1, 3].
- * **Manage the BROski\$ Economy:** Wire the payment gateway to a **Supabase** backend to handle database writes, ensuring that when a user pays, their BROski\$ token balance is updated automatically [2-4].

2. SRE-Level Observability

You move beyond basic logs to mastering professional **Site Reliability Engineering (SRE)** tools. This involves wiring a full observability stack into your infrastructure:

- * **Metrics & Tracing:** Configuring **Prometheus** for scraping metrics and **Tempo** for distributed tracing [3, 5].
- * **Logging & Visualization:** Setting up **Loki** for log aggregation and **Grafana** to create the "Mission Control" dashboards that visualize the heartbeat of all 32+ containers [3, 6].

3. Scaling with Agent Clusters

Module 6 shifts from managing individual agents to orchestrating entire swarms. You utilize the advanced features of the **HyperAgent-SDK** to:

- * **Use the Cluster Builder:** Transition from individual `manifest.json` files to a centralized `cluster.json` deployment configuration [7, 8].
- * **The Graduation Workflow:** Execute the `npm run graduate` command, which allows the system to read your entire cluster's port maps and memory backends for a production-grade launch [8, 9].

4. Advanced Self-Healing

Building on the "Healer Agent" concepts, this module focuses on **"autonomously alive"** infrastructure:

- * You will program agents that can detect service failures via the observability stack and **automatically recover** them, ensuring your system maintains 100% uptime without manual intervention [3, 10].

5. Achieving "BROski Elite" Status

Completion of this module aligns with **Level 5 (BROski Elite 🔥)** in the Agent Unlock System [9]. At this stage, you are no longer just a student but a core contributor capable of:

- * Building **Studio clusters** that plug directly into the HyperCode V2.4 Hyper-Agents-Box [9, 11].
- * Shipping **shipped products** using natural language as your primary interface [2].

Essentially, Module 6 is designed to give neurodivergent creators the tools to scale their vision into a robust, self-sustaining, and monetized AI empire [12, 13].

Would you like a detailed lesson plan for Module 6, including a breakdown of the Stripe integration steps?

In the 10-module curriculum we've outlined, **Module 7: 🐶 Soulful Entities: AI-Native Pets & Memory** focuses on transitioning from building stateless tools to creating AI agents with **persistent personalities and dynamic emotional states** [1, 2].

Drawing from the technical architecture of the **BROskiPets** project and the **HyperAgent-SDK**, here is what happens in this module:

1. Mastering the "State Split" Architecture

Students learn the critical design pattern used for "soulful" agents: separating fast-moving interaction data from permanent identity [3].

- * **Real-Time Layer (Redis):** You will program the agent to track and update "volatile" needs—such as **hunger, energy, happiness, and XP**—every few seconds during interactions [3].
- * **Permanent Layer (IPFS/On-Chain):** You will learn how the pet's core identity and evolution stage are stored permanently, ensuring their "growth" is never lost [3].

2. Implementing the "LLM Brain"

This module dives into the cognitive core of the pets. You will learn to:

- * Use **Qwen2.5:7b** via **Ollama** to power the pet's personality [2].
- * Craft unique **personality profiles** for different rarities, from "Common" pets like **DonutEep** to "Quantum" pets like the **WelshDogEep** [2, 4].
- * Wire the **interaction history** from **Redis** into the LLM's prompt context so the pet "remembers" past conversations with the user [2].

**3. Building Interaction & Evolution Logic

You will move beyond simple chat by building functional "needs" systems:

- * **Feed and Play Mechanics:** Creating endpoints that modify **Redis** state based on user actions [3, 5].
- * **Evolutionary Triggers:** Programming the agent to detect when **XP** thresholds are met, triggering a **metadata update** and an on-chain evolution [3, 6].
- * **Security Guards:** Implementing **VenomEep** layer prompt injection guards to ensure the pet's personality isn't "hijacked" by malicious prompts [6].

**4. Alignment with the Agent Unlock System

Technically, this module aligns with the **Level 3 (Agent Builder)** tier of the SDK's progression system [7]. By completing these tasks, you unlock the ability to build:

- * **Memory-Enabled Agents:** Agents that utilize **Redis** or **Postgres** backends [7].
- * **Multi-Tool Agents:** Entities that can use various tools to satisfy their "needs" or perform tasks for the user [7, 8].

Essentially, **Module 7** is where your code starts to "feel alive," moving from a series of scripts to a "ride or die" AI companion that grows alongside you in the **Hyperfocus Zone** [9, 10].

In **Module 8: Web3 Integration & On-Chain Evolution**, the course moves into the decentralized layer of the ecosystem, teaching you how to transform standard AI agents into **Dynamic NFTs (dNFTs)** that live and evolve on the blockchain.

Based on the technical architecture of the **BROskiPets** project, here is what happens in this module:

**1. Building the "Evolutionary Pipeline"

Students learn to bridge the gap between local AI logic and the Ethereum-compatible blockchain (**Sepolia testnet**) [1, 2]. This involves:

- * **Triggering Evolution:** Wiring the **POST /evolve** API endpoint to the **EEPVengers.sol** smart contract [3].
- * **Metadata Updates:** Learning how to generate new metadata **JSON** files and update the **IPFS CID** (Content Identifier) whenever a pet levels up [1, 4].
- * **The Token URI Shift:** Programming the system to permanently change the **Token URI** on-chain, which updates the pet's visual appearance and traits in real-time across the ecosystem [5].

2. Smart Contract Mastery (EEPVengers.sol**)

This module introduces the **ERC-721** standard with a "Hyper Way" twist:

- * **Dynamic Storage:** Managing per-token URI storage so each of the 78 unique pets can evolve independently [1].
- * **Role-Based Access Control (RBAC):** Implementing separate roles for **Minters**, **Agents**, and **Admins** to ensure no single key controls the entire system [1].
- * **On-Chain State:** Understanding what data is critical enough to stay on-chain (ownership, evolution stage, CID) versus what stays in Redis for speed [4].

3. Integrating Python with Web3

You will move from writing simple Python scripts to using `web3.py` to communicate with the blockchain [3].

- * **On-Chain Calls:** Learning to call contract functions like `evolve()` directly from your FastAPI backend [3].
- * **IPFS Pipeline:** Using Python to handle automated uploads to IPFS (via Pinata) whenever an agent's state changes [1, 3].

4. Security-First Web3 Design

Because blockchain actions are permanent, this module emphasizes the security architecture required for autonomous agents:

- * **Evolution Cooldown:** Implementing the **1-hour rate-limit** per token to prevent damage if an agent key is ever compromised [1, 5].
- * **Pausable Contracts:** Learning how an admin multisig can freeze all mints and evolutions during a system-wide emergency [1].
- * **Reentrancy Guards:** Using standard security patterns to protect state-changing contract functions [1].

The "On-Chain" Objective

The goal of Module 8 is to ensure your creations are **"Autonomously Alive"** beyond your local machine. By the end of this module, you will have built an agent that not only thinks and remembers but also **carves its own history permanently into the blockchain**, represented by an evolving digital soul that you truly own.

"Would you like me to expand this into a lesson plan with a mini-project, such as deploying your first dynamic pet contract to a testnet?"

Based on the 10-module structure developed in our conversation to reflect the full technical depth of the ecosystem, **Module 9:  Security & SRE Observability** is a **Hyper-Pro** level course focused on hardening your AI infrastructure and monitoring it like a professional Site Reliability Engineer (SRE) [1].

In this module, you move beyond building features to ensuring your "autonomously alive" system is secure, resilient, and transparent.

Core Learning Objectives

The primary goal is to master the **defensive architecture** and **real-time monitoring** necessary to run a production-grade agent swarm [2, 3].

Key Skills & Technical Focus

According to the technical standards in the sources, Module 9 covers:

- * **Wiring the Observability Stack:** You learn to integrate the full suite of monitoring tools used in the HyperCode V2.4 engine, including **Prometheus** (metrics), **Grafana** (visualization), **Loki** (logging), and **Tempo** (distributed tracing) [1, 3, 4].
- * **Implementing "VenomEep" Security:** You will build and configure the **VenomEep** layer, a prompt injection guard featuring a 15+ pattern blocklist designed to protect your agents' `chat()` calls from malicious hijacking [2].
- * **Role-Based Access Control (RBAC):** You master the implementation of separate roles—such as **MINTER_ROLE**, **AGENT_ROLE**, and **DEFAULT_ADMIN_ROLE**—ensuring no single compromised key can control the entire ecosystem [2].
- * **Infrastructure Hardening:** Following the ecosystem's security-first philosophy, you learn to implement **Docker isolation** (using `cap_drop: ALL`` and `no-new-privileges``), **Redis authentication**, and **ReentrancyGuards** for smart contracts [2].
- * **Evolution Cooldowns:** You learn to set up **security-first rate-limits**, such as the 1-hour evolution cooldown, to mitigate potential damage in the event of a compromised agent key [2].

The "Hyper-Pro" Outcome

By the end of Module 9, your lab will no longer just be "running"—it will be **"SRE-Grade."** You will be able to look at the **Mission Control Dashboard** or a **Grafana Stack** (typically on port `:3001``) and see the real-time heartbeat, security status, and performance metrics of your entire 32-container stack [4, 5].

Would you like me to expand this into a full lesson plan for Module 9, including a walkthrough for setting up your first Grafana security dashboard?

In the 10-module curriculum we've established, **Module 10: 🚀 Ship, Scale & Graduate** is the final **Hyper-Pro** level stage where you transition from a developer to a producer of "autonomously alive" software [1]. This module represents the peak of the **Agent Unlock System** [2].

Core Objectives

The final mission is to take your vision from a local Docker lab to a **shipped, production-ready product** with a functional economy and automated orchestration [3].

**Key Technical Milestones

According to the sources, the following happens in Module 10:

- * **Production Token Economics:** You integrate **Stripe Checkout** end-to-end to handle real payments for token packs (Starter, Builder, and Hyper packs) [3]. This includes setting up **webhook handlers** and ensuring secure **Postgres database writes** for user balances [3, 4].

- * **The Graduation Workflow:** You master the `npm run graduate` command [5]. This is the "Phase 4" deployment strategy where the system reads your `cluster.json`—generated in the HyperAgent Studio—to launch your entire agent swarm by mapping manifests, ports, and memory backends automatically [5].
- * **SRE Mastery:** You finalize your **observability stack** (Prometheus, Grafana, Loki, Tempo), ensuring you can monitor every heartbeat of your 32-container production environment like a professional Site Reliability Engineer [3, 4].
- * **Self-Healing Finalization:** You ensure your **Healer Agent** (on port `:8008`) is correctly wired to your production metrics so it can autonomously recover services during real-world use [4, 6].

Achieving "BROski Elite" Status

Completion of this module marks your official "graduation" to **Level 5 (BROski Elite 🔥)** [2]. At this level, you have unlocked the full capabilities of the ecosystem:

- * Ability to make **core contributions** to the HyperCode V2.4 engine [2].
- * Mastery of **Studio clusters** for complex swarm management [2].
- * The technical proficiency to deploy agents that are truly **"autonomously alive"** [7].

Essentially, Module 10 is about **removing the final barriers** to entry. You stop asking for permission to build and start shipping your own "Empire in a Box," fully supported by the "ride or die" partnership of your agent crew [8].

Would you like a detailed "Graduation Checklist" for Module 10 to see exactly what counts as a production-ready deploy?